

Стандарт разработки под Android на Kotlin

Основные причины необходимости единого стандарта разработки:

- чтобы код выглядел единообразно по всем модулям и приложениям
- упрощает чтение кода в модулях разработчиками других команд при ревью
- упрощает внесение правок в модули других команд
- облегчает трансфер разработчиков между командами

В компании "Тензор" мы ведем разработку с использованием языка Kotlin и придерживаемся собственного стандарта, основанного на [официальном стандарте Kotlin](#). Любой пункт из этой статьи, вступающий в противоречие с официальным стандартом, имеет больший приоритет.

При написании кода придерживаемся принципов разработки [SOLID](#), [KISS](#), [DRY](#) и [YAGNI](#).

Логирование

Все информативное логирование — только через централизованные утилитные классы (Timber) (с возможностью отключить все одновременно). Не выводим лишних логов, в релизной сборке не пишем дебажных логов.

Исключения

- Пишем обработчики всех исключений.
- Ловим каждый exception отдельно.
- Не используем `finalize`.

Android-модули

Если в модуле есть функционал, переиспользуемый в нескольких приложениях без остального кода модуля, то его следует выносить в отдельный модуль, чтобы оптимизировать процесс сборки приложений.

Не стоит без крайней необходимости создавать слишком маленькие модули, в которых меньше 10 классов.

Во избежание коллизии идентификаторов в приложении все ресурсы в модуле, включая `id` в верстке (`@+id/...`), должны иметь уникальный префикс, например:

```
1 auth_account_chip.xml
2 auth_discovery_host_item.xml
```

В `build.gradle` модуля быть указан атрибут префикса:

```
1 android {
2     ...
3     defaultConfig {
4         ...
5         resourcePrefix "auth_"
6     }
}
```

Комментарии

1. В файле с исходным кодом должны быть прокомментированы классы, интерфейсы, поля, свойства, методы и константы, не помеченные атрибутом доступа `private`.
2. Если в файле с исходным кодом содержится более одной сущности (несколько классов, несколько самостоятельных методов), дополнительно к их описанию необходимо в начале файла добавить общее описание группы сущностей.
3. Все комментарии пишутся на русском языке. Каждое предложение начинается с заглавной буквы. В конце комментария ставится точка.
4. Для документирования java-кода используйте синтаксис [Javadoc](#), а документацию kotlin оформляйте по правилам [Kotlin](#).
5. Когда по названию метода или класса очевидно что он делает, нет сайд-эффектов и дополнительное описание излишне, то в качестве документирования можно добавить `/**@SelfDocumented */`:

о Хорошо:

```
1 private var flag = false
2 /**@SelfDocumented */
3 fun setFlag(value: Boolean) {
4     flag = value
5 }
```

о Плохо:

```
1 private var flag = false
2 /**@SelfDocumented */
3 fun setFlag(value: Boolean) {
4     flag = value
5     burnCityAndKillCitizen() // сайд эффект
6 }
```

В последнем случае должен быть полноценный комментарий:

```
1 private var flag = false
2 /**
3  * Устанавливает значение флага, при этом город будет сожжен и все
4  * жители будут убиты.
5  * @param value @SelfDocumented
6  */
7 fun setFlag(value: Boolean) {
8     flag = value
9     burnCityAndKillCitizen()
10 }
```

6. Не используем аннотацию `{@inheritDoc}`.
7. Используем по месту аннотации из пакета `androidx.annotation.*` в том числе, указываем для каждого метода и конструктора требуемый поток (`@WorkerThread`, `@AnyThread`, `@MainThread`).

Однострочный комментарий

1. Начинается с двойного символа "косая черта" (`//`), после которого ставится один пробел.
2. Расположен в отдельной строке над кодом, который комментируется. Благодаря этому упрощается проверка изменений файла на [GitLab](#): заметны изменения в комментариях и исходном коде.
3. Конец комментария обозначается точкой.
4. Имеет такой же структурный отступ, как и сам код.

Многострочный комментарий

1. Для оформления используйте синтаксис Javadoc.
2. Расположен над кодом, который комментируется.
3. Имеет такой же структурный отступ, как и сам код.

4. Над комментарием оставьте пустую строку.

```
1 fun toBinaryImage(image: Array<Array<Byte>>) {
2     /*
3     Пороговое значение подобрано эмпирически для нашего типа
    изображений.
4     Значения меньше этого значения считаются черным.
5     Увеличение значения привести к "засвечиванию" результата.
6     */
7     val threshold = 100
8
9     //...
10 }
```

Комментарии к коммитам в Git

1. Комментарии должны быть написаны на русском языке.
2. Не допускается отсутствие комментария.
3. Комментарий не должен включать в себя перечисление отредактированных классов или методов, но должны быть указаны изменения в поведении приложения, за исключением рефакторинга.
4. Каждый комментарий должен содержать ссылку на задачу, по которой был оформлен Merge request, кроме автоматически создаваемых (Merge branch ...).

FIXME и TODO

Для временных или вынужденных решений используем TODO-комментарии, либо FIXME. К ним следует добавить ссылку на Задачу/Ошибку, в которой описана проблема и по которой будет выполнено исправление, либо оставить дату, когда будет выполнено исправление (2-3 месяца, не более). В результате разработчик может оперативно понять причину и отследить решение вопроса.

Код с FIXME должен быть исправлен до ближайшего выпуска.

```
1 // TODO: ...
2 // FIXME: ...
```

Дополнительно

Если метод не имплементирован или игнорируется, то пишем так:

```
1 override fun onLoadFailed() = Unit
```

Комментирование ресурсов и строк выполняется так:

```
1 <!--<color name="navigation_primary_color">#ffffff</color>-->
2 <color name="navigation_primary_color">#ebf3f8</color>
```

Настройки стилей Android Studio для Kotlin

Посмотреть и скачать можно по [ссылке](#).